

CS240 Algorithm Design and Analysis
Spring 2026
Problem Set 1

Due: 23:59, March 26, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

Sort the following functions in ascending order of growth.

$$f_1(n) = 2025^{2026^n} \quad (1)$$

$$f_2(n) = 2026^{2025^n} \quad (2)$$

$$f_3(n) = \log_2 8^n \quad (3)$$

$$f_4(n) = n^{\log_2 8} \quad (4)$$

$$f_5(n) = 2027^{2026^{2025}} \quad (5)$$

$$f_6(n) = 2^{1.1 \log_2 n} \quad (6)$$

$$f_7(n) = n^{\log_2 n} \quad (7)$$

$$f_8(n) = 2^{\log_2 \log_2 n} n \quad (8)$$

$$f_9(n) = C_n^{n/2} \quad (9)$$

Hint: $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$.

Solution:

$$(5) < (3) < (8) < (6) < (4) < (7) < (9) < (2) < (1)$$

Problem 2:

Determine the time complexity of the pseudocode below and explain your reasoning.

```
1  def count_triples(arr, threshold):
2      n = len(arr)
3      count1 = 0
4      count2 = 0
5
6      def merge_sort(arr):
7          if len(arr) <= 1:
8              return arr
9
10         mid = len(arr) // 2
11         left = merge_sort(arr[:mid])
12         right = merge_sort(arr[mid:])
13
14         result = []
15         i = j = 0
16
17         while i < len(left) and j < len(right):
18             if left[i] <= right[j]:
19                 result.append(left[i])
20                 i += 1
21             else:
22                 result.append(right[j])
23                 j += 1
24
25         result.extend(left[i:])
26         result.extend(right[j:])
27
28         return result
29
30     arr = merge_sort(arr)
31
32     for i in range(n):
33         for j in range(i + 1, n):
34             k = j + 1
35             while k < n and arr[i] + arr[j] + arr[k] <= threshold:
36                 count1 += 1
37                 k += 1
```

```

38
39     for i in range(n):
40         left = i + 1
41         right = n - 1
42
43         while left < right:
44             if arr[i] + arr[left] + arr[right] <= threshold:
45                 count2 += (right - left)
46                 left += 1
47             else:
48                 right -= 1
49
50     return count1 + count2

```

Solution:

The function `count_triples` consists of three main parts.

1. Merge Sort

The array is first sorted using merge sort. Merge sort divides the array recursively and merges the results. The time complexity is

$$O(n \log n)$$

2. Method 1: Linear Scan

The second counting procedure uses three nested iterations:

- Loop over i : $O(n)$
- Loop over j : $O(n)$
- Inner while loop over k : up to $O(n)$ in the worst case

Thus the complexity of this part is

$$O(n^3)$$

3. Method 2: Two Pointers

The first counting procedure uses a loop over i and a two-pointer scan.

- The outer loop runs n times.
- For each i , the pointers `left` and `right` move at most n steps in total.

Thus the total complexity of this part is

$$O(n^2)$$

4. Overall Complexity

Combining all parts:

$$O(n \log n) + O(n^2) + O(n^3)$$

The dominant term is $O(n^3)$.

Conclusion:

The overall time complexity of the algorithm is

$$O(n^3)$$

Problem 3:

Given a set of n points on the real axis $A = \{A_1, \dots, A_n\}$ ($A_i \in \mathbb{R}$). Please design a greedy algorithm to find **the minimum number of unit-intervals to cover all the given points**. That is to say, each point is covered by at least one interval.

- A **unit-interval** is a closed interval $[l, r]$ where $r - l = 1$.
- A point x is **covered** by an interval $[l, r]$ if and only if $l \leq x \leq r$.

Example: Given $A = \{1.01, 10.1, 3.14, 2.33\}$. Then the minimum number of unit-intervals needed is 3 and $[1.0, 2.0] \cup [2.3, 3.3] \cup [9.5, 10.5]$ are 3 possible unit-intervals covering all points in A .

Note: Your solution should include the following three parts:

- Describe your algorithm in **pseudocode** or **natural language**.
- Proof of correctness of your algorithm using **exchange argument**.
- Time complexity analysis. Your algorithm should run in $o(n^2)$ time.

Solution:

Algorithm 1: Possible Greedy Algorithm

Algorithm 1 Find-Unit-Intervals

```
1: function FIND-UNIT-INTERVALS( $A = \{A_1, \dots, A_n\}$ )
2:   Sort  $A$  in ascending order
3:    $I \leftarrow \emptyset$ 
4:    $R \leftarrow -\infty$ 
5:   for  $i = 1$  to  $n$  do
6:     if  $A_i > R$  then
7:        $I \leftarrow I \cup [A_i, A_i + 1]$ 
8:        $R \leftarrow A_i + 1$ 
9:     end if
10:  end for
11:  return  $I$ 
12: end function
```

The minimum number of unit intervals equals $|I|$.

Proof of Optimality.

Let $[l_1, r_1], \dots, [l_p, r_p]$ with $r_1 < \dots < r_p$ be the greedy solution. Let $[l'_1, r'_1], \dots, [l'_q, r'_q]$ with $r'_1 < \dots < r'_q$ be an optimal solution, and assume $r_1 = r'_1, \dots, r_k = r'_k$ for the largest possible k .

Key Observation. The greedy algorithm places intervals as right as possible. Let x_i be the leftmost point to the right of r_k . Then $l_{k+1} = x_i$ and $r_{k+1} = x_i + 1$. Since the optimal solution must also cover x_i , we have

$$r'_{k+1} \leq x_i + 1 = r_{k+1}.$$

Exchange Argument. Replace $[l'_{k+1}, r'_{k+1}]$ in the optimal solution with $[l_{k+1}, r_{k+1}]$. This exchange can be repeated for $k+1, k+2, \dots$

Conclusion. If $p > q$, then there exists a point x_j with $x_j = l_p$. But $r'_q < l_p = x_j$, contradicting that the optimal solution covers all points. Thus $p \leq q$. Since the optimal solution cannot be worse than greedy, $q \leq p$. Hence $p = q$, and the greedy solution is optimal.

Time Complexity. Sorting takes $O(n \log n)$, and the linear scan takes $O(n)$, so total time complexity is $O(n \log n)$.

Problem 4:

You are driving on a highway represented by an array of fuel stations. You start at the first station. Each station in the array has a number representing how far you can drive from that station before needing to stop at another one for fuel. Assume you always have enough fuel to reach the last station. Your goal is to reach the last station using the smallest possible amount of refuel. Please provide a greedy solution for this scenario.

Example:

- **Input:** `stations = [2, 3, 0, 1, 4]`
- **Output:** 2
- **Explanation:** The minimum number of refuels to reach the last station is 2. Refuel at station 0 to drive to station 1, then refuel at station 1 to drive directly to the last station.

Solution:

Since the last station is always reachable, our goal is to minimize the number of refuels. To achieve this, we should maximize our progress at each step by always refueling at the station that allows us to reach the farthest possible point.

Greedy Algorithm:

- **Step 1:** Define `current_end` to mark the farthest point you can reach with the current number of refuels.
- **Step 2:** Traverse the array and, for each station, calculate how far you can reach. If you reach the `current_end`, it's time to refuel.
- **Step 3:** At each step, choose the station that allows you to reach the farthest possible point, maximizing your gain.
- **Step 4:** Continue this process until you reach or exceed the last station.

Proof:

Assume there exists an optimal solution (OPT) that uses fewer refuels than the greedy algorithm (Greedy). We will now use proof by contradiction to show that this assumption is false and that Greedy is indeed optimal.

We assume that both the greedy algorithm and the optimal solution make the same choices for the first i refuels, i.e., they follow the same path from the start to position p_i . We now analyze the behavior of Greedy and OPT in the $i + 1$ step.

- **OPT:** Let OPT choose to refuel at position p_{opt} , which is the optimal position to refuel from p_i .
- **Greedy:** Greedy will refuel at the farthest possible position from p_i , denoted as p_{greedy} , where $p_{\text{greedy}} = p_i + \max(\text{nums}[p_i])$.

Since the greedy algorithm always chooses the farthest position, we have:

$$p_{\text{greedy}} \geq p_{\text{opt}}$$

This means that the greedy algorithm will either reach or exceed the position chosen by the optimal solution, potentially requiring fewer or at least the same total refuels than OPT. Therefore, Greedy is optimal.

Time Complexity: $O(n)$, since we traverse the array once.

Problem 5:

You are given an $n \times n$ grid, and a procedure $V(i, j)$ that assigns an integer value to each position (i, j) in the grid, where i and j are integers such that $1 \leq i, j \leq n$. Your goal is to find a local minimum (or sink) in the grid (i.e., integers i^*, j^* with $1 \leq i^*, j^* \leq n$ such that for all neighbors (i, j) of (i^*, j^*) in the grid, $V(i^*, j^*) \leq V(i, j)$). The neighbors of (i, j) are $(i - 1, j)$, $(i + 1, j)$, $(i, j + 1)$, $(i, j - 1)$; the elements along the diagonals do not count as neighbors.

Design an algorithm that makes $O(n)$ calls to V . Note that the grid has n^2 nodes.

Solution:

High-level idea. Divide the grid into four quadrants using the middle row and middle column. Let B be the set of *boundary positions*, i.e., those cells that have a neighbor in a different quadrant (the cells on the middle row or the middle column). Compute

$$m \in \arg \min_{b \in B} V(b).$$

If m is not already a local minimum, then m has a strictly smaller neighbor within its own quadrant; following strictly-decreasing neighbors from m reaches a local minimum that cannot cross back to the boundary, so there exists a local minimum strictly inside the quadrant containing m . Hence, after examining B we can recurse into a single quadrant. To make this precise, we strengthen the recursion's specification: given a starting position t , the procedure returns a local minimum whose label is $\leq V(t)$. If $V(m) \leq V(t)$, we recurse into the quadrant containing m and pass m as the new t ; otherwise, we recurse into the quadrant containing t and pass t through.

Algorithm Design.

1. (*Base case*) If the current grid is 1×1 with position t , return t . It is a local minimum.
2. (*Divide*) Split the current grid by its middle row and middle column into four quadrants; let B be the set of boundary positions (middle row $\cup$ middle column).
3. (*Probe boundary*) Find $m \in B$ with minimum value $V(m)$.
4. (*Choose subproblem*) If $V(t) \leq V(m)$, recurse on the quadrant containing t and pass t ; else recurse on the quadrant containing m and pass m .

Pseudocode.

Algorithm 2 FindGridLocalMinimum

Input: An $n \times n$ grid G ; a position t in G ; access to a procedure V assigning an integer to each position in G .

Output: A local minimum of G with label at most $V(t)$

```

1: procedure FINDGRIDLOCALMINIMUM( $G, t$ )
2:   if  $n = 1$  then
3:     return  $t$ 
4:   else
5:     Divide  $G$  as in Figure 1
6:     Let  $B$  represent the boundary positions
7:     Let  $m$  be a position s.t.  $V(m) = \min_{b \in B} V(b)$ 
8:     if  $V(t) \geq V(m)$  then
9:       Let  $G'$  be the quadrant containing  $m$ 
10:      return FINDGRIDLOCALMINIMUM( $G', m$ )
11:    else
12:      Let  $G'$  be the quadrant containing  $t$ 
13:      return FINDGRIDLOCALMINIMUM( $G', t$ )
14:    end if
15:  end if
16: end procedure

```

Correctness. We prove by induction on the current grid size n that FINDGRIDLOCALMINIMUM(G, t) returns a local minimum x of G with $V(x) \leq V(t)$. The base case $n = 1$ is immediate. For the inductive step, let m be the minimum over boundary B . In both branches ($V(t) \leq V(m)$ or $V(t) > V(m)$), we recurse on a quadrant G' with a seed $u \in t, m$ that satisfies $V(u) \leq V(b)$ for all $b \in B$ and $V(u) \leq V(t)$. By the IH, the recursive call returns a local minimum x of G' with $V(x) \leq V(u) \leq V(t)$. Any neighbor of x either lies in G' (so it is not smaller than x) or is in B (whose labels are $\geq V(u) \geq V(x)$), hence x is also a local minimum of the whole grid G .

Time Complexity Analysis: At a grid of side length ℓ , the boundary B has $O(\ell)$ cells; finding $\min_{b \in B} V(b)$ takes $O(\ell)$ evaluations of V . The recursion goes down one quadrant, halving the side length each time, so the total probes are

$$O(n) + O(n/2) + O(n/4) + \dots = O(n).$$

Thus the algorithm makes $O(n)$ calls to V .

Problem 6:

In a city rich with history, there is an old town that was once the heart of the city. However, as time passed, the streets and buildings of the old town fell into disrepair, and the layout became chaotic. The streets in this old town are connected like a linked list, one after another, and each street has a unique number representing its construction period and architectural style. Over the years, these numbers have fallen out of order, leading to inconvenient traffic, dilapidated buildings, and a decline in the quality of life for the residents. To revitalize the old town, the city government has launched the “Old Town Street Reconstruction Plan”. The planners aim to restore the streets to their former glory by arranging them in an orderly sequence according to their numbers, making the layout more logical and facilitating better traffic flow and daily life. Your task is to act as the engineer for this reconstruction project, responsible for reordering these streets.

Given a linked list representing the structure of the streets, where the head of the list is *head*, you are required to use an efficient divide-and-conquer algorithm to sort all the street numbers in ascending order. The reconstructed streets must be sequentially connected, restoring the old town’s orderly beauty. Please return the sorted linked list of streets.

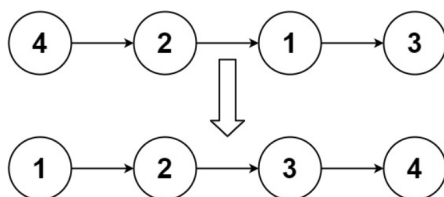


Figure 1: For example

For example, the street numbers of the old town are currently arranged as $[4, 2, 1, 3]$, with the head node $head = 4$. You need to sort them in ascending order as $[1, 2, 3, 4]$.

Solution:

To sort a linked list in ascending order, the most efficient approach is to use Merge Sort. Merge Sort is a divide-and-conquer algorithm with an average time complexity of $O(n \log n)$, which meets the problem’s requirements. Merge Sort is particularly well-suited for linked lists because it allows for splitting and merging in constant time without requiring additional space like arrays.

Solution Steps:

1. Divide and Conquer Strategy: Continuously split the linked list into two halves until each sublist has only one node or is empty. Then, merge these sublists back together to create a sorted list.

2. Finding the Middle of the Linked List: To split the list into two halves, use the fast and slow pointer technique (the fast pointer moves two steps at a time, and slow pointer moves one step). When the fast pointer reaches the end, the slow pointer will be at the middle of the list. Disconnect at the middle to form two separate sublists.

3. Recursively Split the Linked List: Recursively apply the above step to each sublist until the list is reduced to its smallest units (individual nodes).

4. Merge Two Sorted Lists: Use a two-pointer approach to merge two sorted sublists. Create a dummy node as the head of the new merged list, then compare nodes from both sublists and append the smaller one to the merged list. Continue until all nodes are merged.

5. Return the Sorted List: After recursively splitting and merging, return the final sorted linked list.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def sortList(head: ListNode) -> ListNode:
    if not head or not head.next:
        return head
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    mid = slow.next
    slow.next = None
    left = sortList(head)
    right = sortList(mid)
    return merge(left, right)

def merge(left: ListNode, right: ListNode) -> ListNode:
    dummy = ListNode(0)
```

```
tail = dummy
while left and right:
    if left.val < right.val:
        tail.next = left
        left = left.next
    else:
        tail.next = right
        right = right.next
    tail = tail.next
tail.next = left or right
return dummy.next
```
